Master Project: Bridging Vision and Language—A Multi-Image AI Pipeline for Context-Aware NPC Storytelling

Hassan Coulibaly

April 1, 2026

Abstract

Modern interactive media frequently struggles with ludonarrative dissonance, where dynamically driven player actions conflict with static, pre-scripted game narratives. While recent advancements in Vision-Language Models (VLMs) have enabled narrative extraction from single, static images, these approaches lack the temporal and sequential context required to interpret complex gameplay. This project introduces a novel, multi-modal machine learning pipeline designed to generate context-aware, dynamic dialogue for Non-Playable Characters (NPCs) based on sequential gameplay screenshots. The proposed architecture separates visual perception from narrative generation to strictly ground the AI in gameplay reality. First, a VLM sequentially processes in-game images to extract key entities, actions, and environmental states, structuring this data into a chronological scene graph. Second, a Large Language Model (LLM) utilizes this structured data—constrained by established interactive storytelling principles—to generate dramatically meaningful and credible NPC dialogue. By filtering raw visual data into a strict JSON framework before text generation, the pipeline reduces LLM hallucination and arbitrary content creation. This approach demonstrates a scalable method for bridging computer vision and natural language processing, allowing games to dynamically translate visual player events into coherent, immersive lore without sacrificing narrative integrity.

1 Introduction

The landscape of interactive media and video games has evolved from simple, linear progression into expansive, open-world environments where player agency dictates the flow of events. As rendering technologies and physics engines have advanced to create highly realistic simulations, the systems governing in-game storytelling have often lagged behind. In most contemporary open-world games, Non-Playable Character (NPC) dialogue remains heavily pre-scripted. While branching dialogue trees offer the illusion of choice, they are ultimately finite and struggle to account for the emergent, unpredictable actions players take in a physics-driven world.

This master's project implements a prototype *talking agent* pipeline: a batch of time-ordered gameplay screenshots is summarized per frame by a VLM, merged into a single structured *scene graph*, and passed to an LLM that produces short NPC dialogue under explicit grounding constraints. The implementation uses Google's `gemini-2.0-flash` for both the vision and language calls, with extraction forced to `application/json` for machine-parseable state.

2 System Architecture and Implementation

2.1 Design goals

The design follows three principles aligned with the research abstract: (1) **temporal grounding**—images are processed in a user-defined order (e.g., lexicographic filename order as a proxy for capture order); (2) **structured mediation**—raw pixels are never passed directly to the dialogue model; only JSON derived from vision passes through; (3) **constrained narration**—the final prompt instructs the model to avoid inventing entities, locations, or plot beats absent from the scene graph.

2.2 Stage 1: Per-frame visual extraction

For each frame index $i = 0, 1, \ldots$, the pipeline sends the screenshot and a fixed instruction prompt to the VLM. The model must return a JSON object with fields including `location`, `time_of_day`, `player_status`, `recent_action`, `notable_enemies_or_objects`, and `visible_entities` (a list of short strings). Extraction uses a low temperature (0.2) to favor factual stability. Each result is wrapped with metadata: `frame_index`, `source_image`, and `observation`.

2.3 Stage 2: Consolidation into a chronological scene graph

Per-frame observations are merged into one JSON *scene graph* suitable for dialogue generation. Two modes are implemented:

- **LLM consolidation (default).** A second call to the same model merges the ordered frame list into a graph with `timeline_summary`, `environment` (`location`, `time_of_day`, `conditions`), `entities` (with `id`, `name_or_description`, `type`, `first_frame`, `last_frame`), `events` (`frame_index`, `description`, `involved_entity_ids`), and `player_arc` (`status_notes`, `notable_actions`). The consolidator is instructed not to invent content absent from the observations.
- **Simple stitch (`--simple-consolidate`).** A deterministic merge (no extra API call) concatenates actions, collects entity strings, and builds a minimal graph; useful for cost-sensitive runs or ablations.

2.4 Stage 3: NPC dialogue generation

The LLM receives only the consolidated scene graph and a block of *storytelling constraints*: ground claims in the JSON, avoid filling gaps with specific unsupported lore, keep a short campfire recap in second person, and avoid omniscient narration. Dialogue generation uses a higher temperature (0.7) for phrasing variety while the input remains fixed structured state.

3 Codebase, Data Layout, and Reproducibility

The repository is organized as follows (paths relative to project root):

```
talking_agent/
  data/
    test_screenshots/     <- preferred input folder
    extracted_state/      <- *_frames.json, *_scene_graph.json
    generated_stories/    <- *_npc_dialogue.txt
```

```
test_images/            <- legacy samples if test_screenshots is empty
evaluation/
  ground_truth_labels.csv
  human_scoring_template.csv
src/
  main_pipeline.py
  evaluator.py
.env.example
requirements.txt
```

3.1 Execution

From the repository root, after `pip install -r requirements.txt` and setting `GEMINI_API_KEY` in a local `.env` file (see `.env.example`):

```
python src/main_pipeline.py
python src/main_pipeline.py --images-dir path/to/screenshots
python src/main_pipeline.py --simple-consolidate
```

By default, the script uses `data/test_screenshots` if it contains images; otherwise it falls back to `test_images`. Output files are timestamped (UTC) under `data/extracted_state/` and `data/generated_stories/`.

3.2 API keys and version control

Secrets must not be committed. Contributors should copy `.env.example` to `.env` and add the key locally. If a key was ever committed, it should be revoked in the Google AI console and replaced with a new key; historical git commits do not erase exposure.

4 Evaluation

4.1 Structured VLM labels

The script `src/evaluator.py` reads `evaluation/ground_truth_labels.csv` with columns `Image_ID`, `Condition_Tested`, `Ground_Truth`, and `VLM_Prediction` (binary 0/1). It reports precision, recall, and F1 per condition and a macro-averaged F1. Rows should be populated with real labels and model predictions from the experimenter's runs.

4.2 Human scoring

The template `evaluation/human_scoring_template.csv` supports optional Likert-style ratings (e.g., coherence, grounding, engagement) for dialogue or scene-graph quality.

5 Limitations and Future Work

Frame order is user-supplied (e.g., filename order), not inferred from video timestamps. VLM mistakes propagate into the graph and dialogue; consolidation may over- or under-merge events. The pipeline does not yet integrate live game APIs or native scene graphs from engines; extending to engine telemetry would tighten grounding further.